

Programmation Orientée Objet pour le Calcul Scientifique : Part I Survival kit

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- Outils et Langage (Tool)
- Orienté objet (OO)
- Gestion de la mémoire (Mem)
- **Programmation statique (Static)**

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 11 premier flirt avec la programmation statique

Programmation statique qu'est-ce que c'est ? Les lignes de codes écrites sont "exécutées" :

- non pas à l'exécution du programme
- mais pendant la compilation

Le plus simple : les variables statiques

```
int i = 3; // assignment done at compile time

int main() {
    std::cout << " i =" << i << std::endl;
}
```

Item 11 premier flirt avec la programmation statique

Mot clé **static** : membre de classe et non d'objet 1/2

```
struct A{
    static int i;
};
int A::i = 1;

int main(){
    std::cout << A::i << std::endl; // no need to have an object to access i
    A a;
    A a2;
    std::cout << a.i << std::endl; // = 1
    std::cout << a2.i << std::endl; // = 1
    // i is the same in all object of class
    a.i += 2;
    std::cout << a2.i << std::endl; // = 3 !
}
```

Item 11 premier flirt avec la programmation statique

Mot clé **static** : membre de classe et non d'objet 2/2

```
// Since C++17 inline authorises in-class initialization
struct A{
    inline static int i = 1;
};

int main(){
    std::cout << A::i << std::endl; // no need to have an object to access i
    A a;
    A a2;
    std::cout << a.i << std::endl; // = 1
    std::cout << a2.i << std::endl; // = 1
    // i is the same in all object of class
    a.i += 2;
    std::cout << a2.i << std::endl; // = 3 !
}
```

Item 11 premier flirt avec la programmation statique

Mot clé **constexpr** : indique qu'une expression peut être évaluée à la compilation

- cette expression se comportera comme une expression constante
- `constexpr` peut porter sur une variable ou une fonction, voir :
 - <https://en.cppreference.com/w/cpp/language/constexpr>
 - https://en.cppreference.com/w/cpp/language/constant_expression.html

```
constexpr int i = 5;  
  
constexpr int f() {  
    return i+2;  
}
```

Item 11 premier flirt avec la programmation statique

Les expressions constantes sont nécessaires dans les deux cas de figures suivants :

- les paramètres des template (voir item suivant)
- les tailles de tableaux statiques

```
int static_array[constant_expression];
```

Item 11 premier flirt avec la programmation statique

les tableaux statiques : sont des tableaux à la C

```
int array0 [3]; // uninitialized array
int array[3] = {0,1,2};
std::cout << array[0] << std::endl; // array starts at index 0
for (int i = 0; i < 3 ; ++i){
    std::cout << "array["<< i << "] = " << array[i]<< std::endl;
}
```

- La taille est connue statiquement, ie. à la compilation et ne peut pas changer.
- Un dépassement de tableau est une erreur silencieuse (=danger)

```
int array[3] = {0,1,2};
array[3] = 0; //ERROR, memory corruption
```

Item 11 premier flirt avec la programmation statique

les tableaux statiques : sont des tableaux à la C

- leur taille peut être une expression constante (fonction ou variable constexpr)

```
constexpr int i = 5;

constexpr int f() {
    return i+2;
}

int main(){
    int array[f()];
    for (int i = 0; i < f() ; ++i){
        array[i] = i;
    }
}
```

Item 11 premier flirt avec la programmation statique

les tableaux statiques : sont des tableaux à la C

- ces tableaux peuvent être multi-dimensionnels :

```
<type> name [slow-index]...[fast-index]
```

```
int multi_array[2][3] = {    {0,1,2},  
                           {3,4,5}  
};  
for (int i = 0; i < 2; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        std::cout << multi_array[i][j] << std::endl;  
    }  
}
```

L'ordre de stockage est la transposée de celui utilisé en Fortran.

Exo 11.1 Construire, remplir et afficher un tableau de taille [2][2][3].

Item 12 : entrée dans l'univers des template

Paramètres template : la face immergée de l'iceberg...

- **Fonctions Template**
 - abstraction du type des arguments
- **Classes Template**
 - un modèle de classe

Item 12 : entrée dans l'univers des template

Fonctions Template 1/4

- commençons par un exemple

```
void f_int(int a) { std::cout << a << std::endl;}  
void f_double(double a) { std::cout << a << std::endl;}  
void f_char(char a) {std::cout << a << std::endl;}
```

Un paramètre template permet de mutualiser du code pour différent type de données

```
template <typename T>  
void f(T a) { std::cout << a << std::endl;}  
int main(){  
    f(3);  
    f(3.14);  
    f('a');  
}
```

Item 12 : entrée dans l'univers des template

Fonctions Template 2/4

- Syntaxe : on peut écrire indifféremment `typename` ou `class`

```
template <class T>  
void f(T a) { std::cout << a << std::endl;}
```

Exo 12.1 écrire une fonction template qui incrémente son paramètre (passé en entrée/sortie). L'appeler pour différents types numériques (entiers ou flottants).

Item 12 : entrée dans l'univers des template

Fonctions Template 3/4 : danger des templates

- une fonction template ne peut pas fonctionner pour n'importe quel type
 - impossible de spécifier les types requis (avant les concepts de C++20)

```
template <typename T>
void f(T& a) { ++a; }

int main(){
    int i = 0;
    f(i); // OK
    bool b = true;
    f(b); // KO, ++ operator not available for bool type
}
```

Item 12 : entrée dans l'univers des template

Fonctions Template 3/4 : introduction des concepts

- `#include <concepts>` introduits en C++20
- permettent d'apporter des restrictions sur le type passé en argument du template

```
#include <concepts>
template <typename T>
requires std::incrementable<T>
void f(T& a) { ++a; }

int main(){
    int i = 0;
    f(i); // OK
    bool b = true;
    f(b); // KO, ++ operator not available for bool type
}
```

Item 12 : entrée dans l'univers des template

Classes Template 1/13

- commençons par un exemple

```
class IntData {  
    int m_i;  
public:  
    void print();  
    void add(int a);  
};  
class FloatData {  
    float m_f;  
public:  
    void print();  
    void add(float a);  
};
```

- Peut-on mutualiser ?

Item 12 : entrée dans l'univers des template

Classes Template 2/13

- commençons par un exemple : réécriture template

```
template <typename T>
class DataT {
    T m_i = T{}; // default value of type T
public:
    void print() {}
    void add(T a) {}
};
int main(){
    DataT<int> int_data;
    DataT<float> float_data;
}
```

- Attention la classe `DataT` n'est compilée que lorsqu'elle est déclarée.

Item 12 : entrée dans l'univers des template

Classes Template 3/13

un modèle de classe qui devient une classe à sa déclaration

- les classe template peuvent contenir plusieurs paramètres template
- les paramètres templates peuvent être des types ou des `int` ou des `bool`
- les paramètres templates peuvent être (partiellement) spécialisés
- la déclaration et la définition doivent être accessibles à l'instanciation de la classe

Item 12 : entrée dans l'univers des template

Classes Template 4/13

- les classe template peuvent contenir plusieurs paramètres template

```
template <typename T, typename U>
class Pair {
    T m_a = T{}; // default value of type T
    U m_b = U{}; // default value of type U
public:
    void print() {}
    void add(T a, U b) {}
};
int main(){
    Pair<int, float> int_data;
    Pair<float, double> float_data;
}
```

Item 12 : entrée dans l'univers des template

Classes Template 5/13

- les paramètres templates peuvent être des types ou des `int` ou des `bool`

```
template <typename T, int N>
class ArrayT {
    T m_i [N] = {}; // empty array
public:
    void print() {}
    void add(T* a) {}
};
int main(){
    ArrayT<int,2> int_data;
    ArrayT<float,3> float_data;
}
```

Item 12 : entrée dans l'univers des template

Classes Template 6/13

- les paramètres templates de la classe peuvent être (partiellement) spécialisés

```
template <typename T>
class DataT {
    T m_data = T{}; // default value of type T
public:
    void print();
    void add(T a, U b);
};
template <> // specialization: no add method
class DataT<bool>{
    T m_data = T{}; // default value of type T
public:
    void print();
};
```

Item 12 : entrée dans l'univers des template

Classes Template 7/13

- les paramètres templates de la classe peuvent être (partiellement) spécialisés

```
template <typename T, typename U>
class Pair {
    T m_a = T{}; // default value of type T
    U m_b = U{}; // default value of type U
public:
    void print() {}
    void add(T a, U b) {}
};
// specialization
template <typename U>
class Pair<int,U>{
    // specific behavior for a pair which first arg is an int...
};
```

Item 12 : entrée dans l'univers des template

Classes Template 8/13

- les méthodes de la classe peuvent être spécialisées (pas partiellement)

```
template <typename T, typename U>
struct Pair {
    void print() {std::cout << "Pair<T,U>"<< std::endl;}
};
template <> // specialization
void Pair<int,int>::print() {std::cout << "Pair<int,int>"<< std::endl;}

int main(){
    Pair<int,double> p1;
    Pair<int,int> p2;
    p1.print();
    p2.print();
}
```

Item 12 : entrée dans l'univers des template

Classes Template 9/13

- la déclaration et la définition doivent être accessibles à l'instanciation de la classe

```
template <typename T> // -- A.h --
struct A {
    T m_ = T{};
    void print(); }; // declaration only
#include "A.h" // -- A.cpp --
template <typename T>
void A<T>::print() {} // definition (here empty for concision)
// -- main .cpp --
#include "A.h"
int main(){
    A<int> a;
    a.print(); // ERROR: undefined reference to 'A<int>::print()'
}
```

Item 12 : entrée dans l'univers des template

Classes Template 10/13

- la déclaration et la définition doivent être accessibles à l'instanciation de la classe
 - lorsque l'objet `a` est créé, la **définition** et la **déclaration** de l'ensemble de la classe doivent être **visibles** : dans un `.h` inclus ou dans le même fichier
 - les template sont **déclarés et définis dans des fichiers `.h`**
 - Les template peuvent être définis dans un `.cpp` s'ils ne sont utilisés qu'à l'intérieur de ce `.cpp`.
 - si l'on souhaite séparer définition et déclaration dans deux fichiers : faire deux `.h` et include la déclaration dans la définition

Item 12 : entrée dans l'univers des template

Classes Template 11/13

- la déclaration et la définition doivent être accessibles à l'instanciation de la classe

```
template <typename T> //-- A.h --
struct A {
    T m_ = T{};
    void print(); // declaration only
};
template <typename T>
void A<T>::print() {} // definition within the A.h file
// main .cpp
#include "A.h"
int main(){
    A<int> a;
    a.print(); // OK
}
```

Item 12 : entrée dans l'univers des template

Classes Template 12/13

- la déclaration et la définition doivent être accessibles à l'instanciation de la classe

```
template <typename T> //-- A_impl.h --
void A<T>::print() {} // definition within the another.h file
//-- A.h --
template <typename T>
struct A {
    T m_ = T{};
    void print();}; // declaration only
#include "A_impl.h"
//-- main .cpp --
#include "A.h"
int main(){
    A<int> a;
    a.print(); // OK
}
```

Item 12 : entrée dans l'univers des template

Classes Template 13/13

- Exo 12.2 faites fonctionner ce programme qui affiche le contenu du tuple

```
#include "Tuple.h"

int main(){
    Tuple<int,int,int> my_tuple {1,2,3};
    my_tuple.print();
    Tuple<int,double,int> my_tuple2 {1,1.2,3};
    my_tuple2.print();
    return 0;
}
```

- le `Tuple<int,int,int>` affiche "Made of int"
- séparer la déclaration et la définition de votre tuple dans deux fichiers.